

Basics of Python

Agenda:

- Write our first Program in Python
- Python Data Types & Comments
- Variables, Keywords & Identifiers in Python
- Python Input
- Type Conversion in Python
- Literals in Python
- Operators in Python

Our First Python Program

Print Function

Text

```
print("Hello World")
```

```
Hello World
```

```
print('Hello World')
```

```
Hello World
```

```
print('Inception BD')
```

```
Inception BD
```

```
print(Inception BD)
```

```
Cell In[4], line 1
    print(Inception BD)
      ^
```

```
SyntaxError: invalid syntax. Perhaps you forgot a comma?
```

Numbers - Integer, Float, Boolean

```
print(9)
```

```
9
```

```
print(9.9)
```

9.9

```
print(True)
```

True

```
print(False)
```

False

Multiple Data Type

```
print("Hello", 1,2,3,1,4, True)
```

Hello 1 2 3 1 4 True

Define Separator into Print function

```
print("Hello", 1,2,3,1,4, True, sep="-")
```

Hello-1-2-3-1-4-True

```
print("Bappy")
```

```
print("Inception")
```

Bappy

Inception

Define End into Print function

```
print("Bappy", end="-")
```

```
print("Inception")
```

Bappy-Inception

Data Type in Python & Comments

1. Integer
2. Float (decimal)
3. Boolean
4. String (text)
5. Complex
6. List
7. Tuple
8. Sets
9. Dictionary

Comments

```
# This is a single line comment, thsi print function prints a sentence  
print("My name is Riduan")
```

```
My name is Riduan
```

```
"""  
This is a multiline Comment  
thsi print function prints a sentence  
"""  
print("My name is Riduan")
```

```
My name is Riduan
```

1. Integer

```
print(2)
```

```
2
```

```
print(20000000)
```

```
20000000
```

```
type(2)
```

```
int
```

2. Float (Decimal)

```
print(3.3)
```

```
3.3
```

```
type(3.3)
```

```
float
```

3. Boolean

```
# Python is case sensetive language
```

```
print(True)
```

```
True
```

```
type(True)
```

```
bool
```

4. String

```
print("Inception BD")  
Inception BD  
type("Inception BD")  
str
```

5. Complex

```
print(5+7j)  
(5+7j)  
type(5+7j)  
complex
```

6. List

```
print([1,2,3])  
[1, 2, 3]  
type([1,2,3])  
list
```

7. Tuple

```
print((1,2,3))  
(1, 2, 3)  
type((1,2,3))  
tuple
```

8. Sets

```
print({1,2,3})  
{1, 2, 3}  
type({1,2,3})  
set
```

9. Dictionary

```
print({"Name":"Riduan","Age":25})
```

```
{'Name': 'Riduan', 'Age': 25}
type({"Name": "Riduan", "Age": 25})
dict
```

Variables, Keywords & Identifiers in Python

Variables

```
a = 2 # Integer
print(a)
b = 3.3 # Float
print(b)
c = True # Boolean
print(c)

2
3.3
True

a
2

type(a)
int

name = "Riduan"
print("Welcome", name)

Welcome Riduan

a = 2
b = 5

print(a+b)

7

# Dynamic Typing
a = 7

# Static Typing
int a = 7

Cell In[55], line 5
    int a = 7
        ^
SyntaxError: invalid syntax
```

```
type(a)
```

```
int
```

```
a = 7
```

```
a = "Riduan"
```

```
print(a)
```

```
Riduan
```

```
# Dynamic Binding
```

```
a = 5
```

```
print(a)
```

```
a = "Riduan"
```

```
print(a)
```

```
5
```

```
Riduan
```

```
# Static Binding
```

```
int a =7;
```

```
str a = "Riduan";
```

```
Cell In[61], line 2
```

```
int a =7;  
      ^
```

```
SyntaxError: invalid syntax
```

```
a = 1
```

```
b = 2
```

```
c = 3
```

```
print(a)
```

```
print(b)
```

```
print(c)
```

```
1
```

```
2
```

```
3
```

```
a = 1
```

```
b = 2
```

```
c = 3
```

```
print(a, b, c)
```

```
1 2 3
```

```

a,b,c = 1,2,3
print(a, b, c)
1 2 3
a = b = c = 4
print(a, b, c)
4 4 4

```

Keywords

39 keywords

{	False	async	def	for	is	raise
	None	await	del	from	lambda	return
	True	in	elif	global	nonlocal	try
	and	break	else	if	not	while
	as	class	except	import	or	with
	assert	continue	finally	in	pass	...

Identifiers

```

name = "Riduan"
print(name)
Riduan
# You can't start with digit
lname = 'Riduan'

```

Cell In[67], line 3
 lname = 'Riduan'
 ^
 SyntaxError: invalid decimal literal

```

name1 = 'Riduan'
# You can't use any special Character except _
*name = "RIduan"
    Cell In[69], line 3
      *name = "RIduan"
      ^
SyntaxError: starred assignment target must be in a list or tuple

_name = "Riduan"

```

Python Input

Static - Calender, Clock

Dynamic - Youtube, Facebook

```

input()
'Riduan'
var = input()
print(var)
82
type(var)
str
# Type Casting
var = float(var)
type (var)
float
input("Enter you name: ")
'Riduan Aziz'

```

Simple Calculator using input

```

a = float(input("Enter the first number: "))
b = float(input("Enter the second number: "))

result = a + b
print(result)

```

9.0

Type Conversion in Python

1. Implicit - Internally by Python
2. Explicit - By the Programmer

Implicit

```
a = 5 + 5.5
```

```
print(a)
print(type(a))
```

```
10.5
<class 'float'>
```

Explicit

```
b = str(5) + "5"
```

```
print(b)
print(type(b))
```

```
c = 5 + int("5")
```

```
print(c)
print(type(c))
```

```
55
<class 'str'>
10
<class 'int'>
```

Literals in Python

```
a = 3
print(type(a))
```

```
<class 'int'>
```

```
a = 0b1010 # Binary Literals
b = 100 # Decimal Literals
c = 0o310 # Octal Literals
d = 0x12c # Hexadecimal Literals
```

```
print(a)
print(b)
print(c)
print(d)
```

```
10
100
200
300
```

Float Literals

```
float_1 = 10.5
float_2 = 1.5e2 # 1.5 * 10^2
float_3 = 1.5e-3 # 1.5 * 10^-3
```

```
print(float_1)
print(float_2)
print(float_3)
```

```
10.5
150.0
0.0015
```

Complex Literals

```
x = 3.14j
```

```
print(type(x))
```

```
<class 'complex'>
```

String Literals

```
string = 'This is Python'
strings = "This is Python"
char = "C"
multiline_str = """This is a multiline string with more than one line
code."""
unicode = u"\U0001f600\U0001f606\U0001f923"
raw_str = r"raw \n string"
```

```
print(string)
print(strings)
print(char)
print(multiline_str)
print(unicode)
print(raw_str)
```

```
This is Python
```

```
This is Python
```

```
C
```

```
This is a multiline string with more than one line code.
```

```
😄😄
```

```
raw \n string
```

```
# Boolean Literals
```

```
# True = 1
```

```
# False = 0
```

```
a = True + 4
```

```
b = False + 10
```

```
print("a:", a)
```

```
print("b:", b)
```

```
a: 5
```

```
b: 10
```

```
# None Literals
```

```
a = None
```

```
b = 2
```

```
c = 5
```

```
print(a, b, c)
```

```
None 2 5
```

Operators in Python

- Arithmetic Operators
- Relational Operators
- Logical Operators
- Bitwise Operators
- Assignment Operators
- Membership Operators

Arithmetic Operators

```
print(3+4) # Addition
```

```
print(3-4) # Subtraction
```

```
print(3*4) # Multiplication
```

```
print(3/4) # Division
```

```
print(3//4) # Floor Division
```

```
print(3%4) # Modulus
```

```
print(3**4) # Exponentiation
```

```
7
```

```
-1
```

```
12
```

```
0.75
```

```
0
```

```
3
```

```
81
```

Relational Operators

```
print(4>5) # Grater Than
print(4<5) # Less Than
print(5>=3) # Greater Than or Equal To
print(4<=5) # Less Than or Equal To
print(4==5) # Equal To
print(4!=5) # Not Equal To
```

```
False
True
True
True
False
True
```

Logical Operators

```
print(0 and 1) # AND
print(0 or 1) # OR
print(not 1) # NOT
print(not 0) # NOT
```

```
0
1
False
True
```

Bitwise Operators

```
print(2 & 3) # Bitwise AND
print(2 | 3) # Bitwise OR
print(2 ^ 3) # Bitwise X-OR
print(~3) # Bitwise NOT
print(4 >> 2) # Right Shift
print(5 << 2) # Left Shift
```

```
2
3
1
-4
1
20
```

Assignment Operators

```
# Increment Operator
a = 2
a += 1
print(a)
```

```
# Decrement Operator
```

```
b = 5
```

```
b -= 1
```

```
print(b)
```

```
3
```

```
4
```

Membership Operators

```
# in/not in
```

```
print('B' in "Bangladesh")
```

```
True
```